

Homigo Care Admin Panel Blueprint

- [Homigo Care — Admin Panel Blueprint](#)
 - [0. Scope & Placement](#)
 - [1. Tech Stack \(Admin-Specific\)](#)
 - [2. Access Control — 5 Admin Levels \(unchanged from source, restated for this stack\)](#)
 - [3. Full CRUD Content Management \(CMS\) — Your Explicit Requirement](#)
 - [4. Payments Module \(Manual System\)](#)
 - [5. Verification & Assessment Oversight](#)
 - [6. Bookings, Safety & Emergency Modules](#)
 - [7. Analytics Dashboard](#)
 - [8. Admin User Management](#)
 - [9. Security Notes Specific to the Admin Panel](#)

Homigo Care — Admin Panel Blueprint

Web-Only Control Center — Build Specification for AI Coding Agents

Prepared from the same 13 source documents · Consolidated 2026

0. Scope & Placement

The Admin Panel is a **web-only** application (never native mobile — this matches your instruction: apps run on Android/iOS, the admin panel runs on the web). It is a **separate Next.js project** (homigo-admin) from the public homigo-web site, sharing the same backend (Firebase + Supabase) but with its own login, its own domain/subdomain (e.g. admin.homigocare.pk), and hosted on **Vercel** exactly like the public site.

Golden rule (repeated from the main blueprint because it matters most here): zero fake data. The Admin Panel ships completely empty. No demo professionals, no demo bookings, no demo revenue charts. Every screen shows a genuine empty state until real people use the real app. The admin’s very first job after deployment is to manually create/approve the first real records for testing — that is the QA process, not a shortcut around it.

1. Tech Stack (Admin-Specific)

Layer	Technology
Framework	Next.js 14+ (App Router) + TypeScript, same repo family as the public web app so components/design tokens are reusable
Styling	Tailwind CSS, Desktop-first (admins work on laptops/desktops; make it responsive down to tablet, but do not need the 3-shell mobile treatment from the public site)
Auth	Supabase Auth with a dedicated admin role claim (separate from patient/professional Firebase Auth) + optional TOTP 2FA
Data	Supabase Postgres (same database as the public app, admin role has elevated RLS policies) + Firebase Firestore read-access for live/real-time views (live map, active bookings)
File access	Cloudinary signed URLs (view-only, generated per request via a Vercel serverless function) so CNIC/degree photos are never publicly exposed
Hosting	Vercel, auto-deploy from the homigo-admin GitHub repo
Email	EmailJS for admin-triggered emails (approval/rejection notices, password resets, support replies)

2. Access Control — 5 Admin Levels (unchanged from source, restated for this stack)

Role	Can Access
Super Admin	Everything, including creating/removing other admin accounts and editing commission/pricing rules
Verification Admin	Professional document review, 3-phase assessment oversight, approve/reject professionals
Support Admin	Chat/ticket queue, dispute resolution, CMS content

Finance Admin

Emergency Admin

Enforce this at the database layer with Supabase RLS policies keyed to each admin’s role claim — **not** just hidden UI buttons, so a Support Admin literally cannot query the finance tables even via direct API call.

3. Full CRUD Content Management (CMS) — Your Explicit Requirement

”جو بھی سپورٹس نمبر جو بھی ای میلز... ایڈمن پینل سے پورا ایڈیٹ کا ایشن ہونا چاہیے... ہر چیز ہم ایڈمن پینل سے ایڈیٹ اور ایڈٹ وغیرہ سب کچھ کر سکیں“

Nothing about support contact info, About Us, or legal content may be hardcoded in the Android/iOS/Web apps. All of it lives in the cms_content Supabase table and the apps fetch it at runtime. The Admin Panel gives full Create/Read/Update/Delete on:

Module	Fields Admin Can Add/Edit/Delete
Support Center	Support phone number(s), WhatsApp number, support email, live-chat availability hours, office address, emergency helpline numbers list (1122/15/115/1099/Homigo’s own)
About Us	Mission/vision text, founder note, company story, team section, image uploads (via Cloudinary)
Legal Pages	Terms & Conditions, Privacy Policy, Refund Policy — rich-text editor, versioned (old versions kept for compliance)
FAQ	Add/edit/delete/reorder question-answer pairs, per category (Patient FAQ / Professional FAQ)
Announcements / Banners	In-app banner text, target audience (all/patients/professionals/specific city), start/end date, optional image
Service Categories & Pricing	Edit the pricing table from \$3.6 of the main blueprint, add new service categories, enable/disable a category app-wide
Role Requirements	Add a brand-new professional role or edit an existing role’s required documents/tests without a code deployment (this directly powers the dynamic registration engine in \$2.2 of the main blueprint)
Push Notification Templates	Compose and send one-off or scheduled notifications, choose priority tier (mandatory/important/optional) and audience segment

Every module here follows the same pattern: **List view → Add New → Edit → Delete (with confirmation) → Change history**. No content is ever “code-only.”

4. Payments Module (Manual System)

Screen	Function
Payment Accounts	Super/Finance Admin adds unlimited receiving accounts: JazzCash number + account title, Easypaisa number + account title, Bank name + IBAN + account title. Each can be toggled active/inactive instantly (e.g. if a JazzCash number needs rotating).
Incoming Payment Verification Queue	Every booking sitting in payment_pending_verification shows: patient name, amount, method, transaction ID entered by patient, and the uploaded screenshot (via signed Cloudinary URL). Admin clicks Confirm Paid or Reject/Ask Patient to Resend , entering a note either way.
Professional Payout Queue	List of professionals with pending earnings, filter by daily/weekly/monthly/on-demand request, minimum PKR 500 rule enforced by the UI. Admin transfers manually outside the system, then clicks Mark Paid , which logs the payout in wallet_transactions/a payouts table with timestamp and admin ID for audit.
Commission & Revenue Reports	Auto-calculated from real completed bookings only (10%/8% rules from \$3.6 applied automatically) — genuinely PKR 0 until real bookings exist, per the zero-fake-data rule.

This entire module is intentionally manual by your instruction — no JazzCash/Easypaisa API integration yet; the abstraction is left open so a future “Auto-Verify Payments” toggle can be added without restructuring these tables.

5. Verification & Assessment Oversight

- **Document Queue:** every new professional sign-up lands here with their uploaded documents (CNIC, degree, license, affiliation letter — set dynamically per their role from `role_requirements`). Admin views each document via a signed Cloudinary URL, approves or rejects per document (not just per applicant), with a mandatory rejection reason that’s emailed to the applicant via EmailJS.
- **Assessment Results:** view every attempt’s score per phase, manually trigger a re-assessment if a complaint or rating drop requires it (per §2.3 of the main blueprint), and manage the 200+ question bank (add/edit/retire questions per phase).
- **Badge Management:** view/override a professional’s badge tier (Certified/Excellence/Elite) if a manual review warrants it.

6. Bookings, Safety & Emergency Modules

Module	Function
Live Bookings Map	Real-time view (Firestore-driven) of every active booking’s GPS position — this is the one screen that legitimately needs live data the moment real bookings start flowing
All Bookings	Full searchable/filterable table (by status, date, professional, patient, service type) — empty until real bookings exist
SOS / Emergency Alerts	Dedicated Emergency Admin view: incoming SOS alerts sound an audible browser alert, show live GPS, one-click “Call Professional” / “Call Patient” (via tel: links or an integrated calling widget), auto-drafted incident report the admin fills in and saves to that patient’s <code>access_logs/incident history</code>
Disputes	Patient-vs-professional dispute queue, evidence attachments (via Cloudinary), resolution notes, refund trigger (feeds back into the manual payments module)
Reviews Moderation	Every review AND every professional reply sits in a moderation queue before publishing (per §3.8 of the main blueprint) — approve/edit-for-policy-violation/reject

7. Analytics Dashboard

Real, computed-from-real-data charts only (bookings over time, revenue by service category, top-rated professionals, city-wise demand, referral leaderboard) — every chart component should be built against the live query from day one and simply **render an empty/zero state gracefully** rather than being wired to sample data during development.

8. Admin User Management

Super Admin can add/remove/deactivate other admin accounts and assign one of the 5 roles from §2. Every admin action anywhere in the panel is written to an `admin_audit_log` table (who did what, when, to which record) — this is your accountability layer since the payments and verification systems are manual and human-operated.

9. Security Notes Specific to the Admin Panel

- Separate Supabase admin auth role — an admin account is never the same row/table as a patient or professional account.
- Recommend TOTP-based 2FA (Supabase Auth supports this) for all 5 admin roles, mandatory for Super Admin and Finance Admin at minimum.
- Login alert email via EmailJS on every admin login from a new device/browser.
- All document/CNIC viewing goes through short-lived signed Cloudinary URLs generated by a Vercel serverless function — the admin panel’s own database never stores a public file URL.
- Full audit log (§8) doubles as your security review trail.